AI OPERATOR IN A DIGITAL TWIN ENVIRONMENT

Oluwarotimi Arijeloye

12.3.2026

Table of Contents

Table of Contents 2

1.0 INTRODUCTION 3

1.1 My Understanding of Explainable AI and the Techniques Involved 3

2.0 Methodology 4

2.1 Data Collection 4

2.2 Model Training and Evaluation 4

2.3 XAI Methods - SHAP and LIME plots 5

2.4 Dashboard Design 5

3.0 Conclusion 5

4.0 Challenges and Limitations 6

5.0 Declaration of AI Usage 6

6.0 References 6

7.0 Implementation Procedures 7

1.0 INTRODUCTION

This proof-of-concept addresses machine failure prediction and energy efficiency detection within a production-line (digital twin). An AI-powered operator assistant in a digital twin environment generates real-time predictions, risk alerts, and actionable recommendations such as adjusting valve settings or anticipating equipment failures while the human operator retains full decision-making authority. The system integrates core capabilities: a predictive engine for assessing failure risk, an interpretability layer using SHAP and LIME to justify every prediction transparently and to evaluate potential interventions confidently before execution. The result from this work can be further improved to design trustworthy, explainable AI assistant for large-scale production environments with actual sensor data.

1.1 My Understanding of Explainable AI and the Techniques Involved

My first encounter with explainable AI occurred during our initial meeting, and it left a considerable impression on me. That introduction prompted me to engage more seriously with the subject, leading me to study various articles and research papers in order to develop a more thorough and informed understanding of the field.

One of the aspects that resonated with me most profoundly was the role of human-in-the-loop (HITL) within XAI. It reflects a principled recognition that AI systems must not only perform well, but must also be accountable and transparent qualities that are essential for building genuine trust among users and stakeholders alike. As my understanding deepened, I began to appreciate the structured nature of XAI as it addresses interpretability in black-box algorithms through a well-defined framework, categorizing methods by their purpose pre-model, in-model, or post-model their scope, whether local or global, and their usability, distinguishing between model-agnostic and model-specific approaches.

The practical application of LIME and SHAP to the moral dataset was a particularly instructive experience. According to [1] comparing the performance outcomes of these two methods offered valuable insight into their respective strengths and limitations. It became evident that while each method contributes meaningfully on its own, neither is sufficient in isolation. A more comprehensive understanding emerges when multiple techniques are applied in combination, ensuring that the explanations generated are robust and accessible to all relevant stakeholders.

This experience reinforced for me that explainability is not merely a technical consideration, but a broader commitment to responsible and transparent AI development.

Another concept I found interesting is the contrast between interpretability and explainability. According to [2] providing interpretability and explainability in the system increases the trust of the users and helps in rectifying deficiencies, it adversely affects the performance of the models. The more interpretable the systems are, the lower the performance will be. Consequently, achieving a balance between interpretability and performance is always a challenge. Performance is considered as the accuracy in an out-of-sample prediction of the model with no concern regarding the reason for such an outcome. Conversely, interpretability refers to the ability of the model to convey information in a comprehensible manner to humans.

2.0 Methodology

The steps involved in this task involved downloading datasets from Kaggle – Loading the dataset to a training model, training with XGBoost model, computing SHAP + LIME explainers, evaluating the model and representing everything on a dashboard. A summary of how the data was cleaned, features engineered, selected models (i.e. XGBoost) used to train them, and how performance was evaluated using metrics like accuracy ROC-AUC are shown in section 7.

I intend using another Kaggle dataset (Industrial Machine Predictive Maintenance) to train the model or preferably dataset close to the research project.

2.1 Data Collection

The Predictive Maintenance Dataset (AI4I 2020) was downloaded from Kaggle. It contains an overview of the required dataset source key features (sensor readings, operating conditions), the target variable (failure or efficiency), and I considered the data suitable for simulating a production-line digital twin.

2.2 Model Training and Evaluation

The modeling approach transforming the raw production-line sensor data into a predictive, explainable AI system involved several steps which includes data preprocessing, feature engineering, model selection and model validation.

- *Data Preprocessing*: The dataset was prepared for modelling by cleaning the missing values and removing obvious outliers. Categorical variables (i.e., machine failure) were converted into numerical form as the model does not accept direct string input. The data was normalized or scaled continuous features where needed and splitted the dataset into training and testing sets to avoid data leakage.
- *Feature Engineering*: This step is crucial because explainability depends on interpretable features. So, features were grouped by subsystem (e.g., thermal, mechanical, operational) to support interpretability and ensured all features map to real physical concepts operators understand. I further derived domain-relevant features such as load ratios, temperature deltas, or rolling averages.
- *Model Selection*: XGBoost was selected as the main model due to strong performance on tabular industrial data.
- *Model Validation*: Cross-validation was carried out using error analysis to identify common misclassifications and to ensure robustness.

2.3 XAI Methods - SHAP and LIME plots

A description of how SHAP and LIME were used to generate global and local explanations is shown in section 7.

2.4 Dashboard Design

The saved model was prepared as input for the Streamlit dashboard interface layout which displayed the prediction panel, SHAP/LIME explanation sections, and interactive what-if controls (designed to mimic an operator console). Also, there is a brief assessment of how different operator types (line operator, maintenance engineer, energy manager) would interpret the explanations, and whether the system supports their decision-making needs.

3.0 Conclusion

This project demonstrates how an explainable AI assistant can support safer and more efficient production-line operations by combining predictive modeling with clear, operator-focused explanations. By integrating SHAP, LIME, the system not only predicts failures but also helps operators understand and act on them confidently. With real-time data and a solid understanding

of how the equipment operates, I am confident that a far more suitable and accurate model can be developed, grounded in deeper analysis and more realistic use-case scenarios.

4.0 Challenges and Limitations

Some of the challenges encountered are listed below;

- SHAP wasn't compatible with Python 3.14 and some other dependencies. I worked with a lower version of Python (i.e. 3.11).
- Local deployment works fine as shown in Figure 7 & 8, but I still have challenges debugging what is hindering the Streamlit Cloud deployment depicted in Figure 9 & 10.
- My understanding of SHAP and LIME continues to evolve, and I am working towards the level of depth required to interpret and justify their explanations with full confidence. That said, I am convinced that grounding the analysis in richer, more realistic use case scenarios would significantly enhance its meaning and practical relevance.

5.0 Declaration of AI Usage

- Co-pilot was used for setting up the environment, generating scripts and debugging of the models used.
- Co-pilot was also used for text rephrasing in preparing this report.

6.0 References

[1] S. Bhatnagar, R. Agrawal "Understanding explainable artificial intelligence techniques: a comparative analysis for practical application", Bulletin of Electr. Eng & Inf, Vol. 13, No. 6, December 2024: 4451-4455.

[2] S. Moosavi, M. *et. al*, "Explainable AI in Manufacturing and Industrial Cyber‑Physical Systems: A Survey", Electronics 2024, 13, 3497. https://doi.org/10.3390/electronics13173497

[3] Kaggle Dataset

7.0 Implementation Procedures

The figures shown below are the steps carried out in section 2.

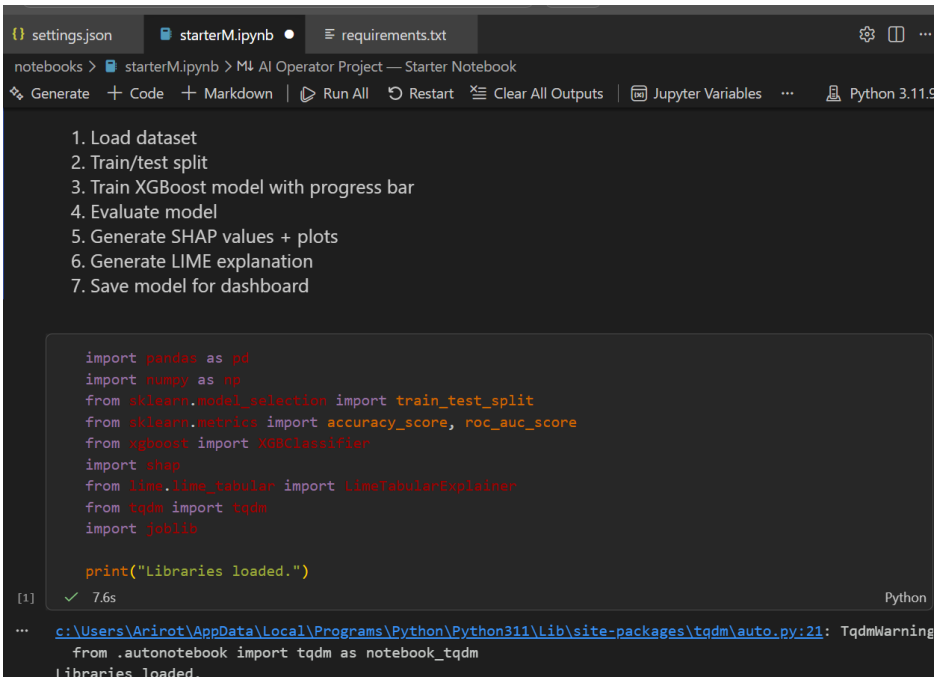

Figure 1: Importing required libraries and dependencies for data preprocessing, feature engineering, model selection and validation.

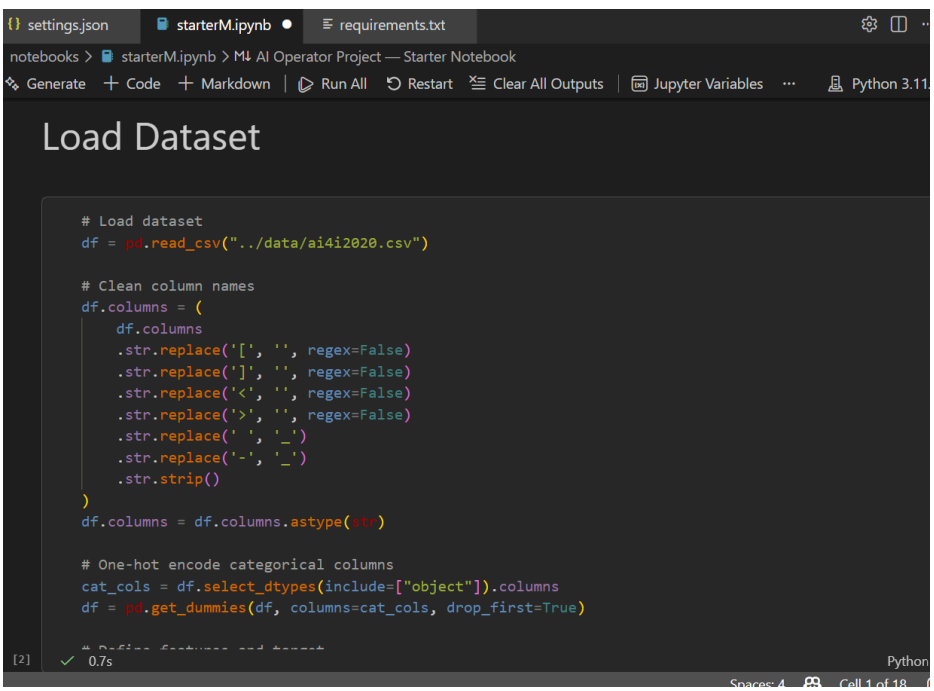

Figure 2: Loading the Dataset

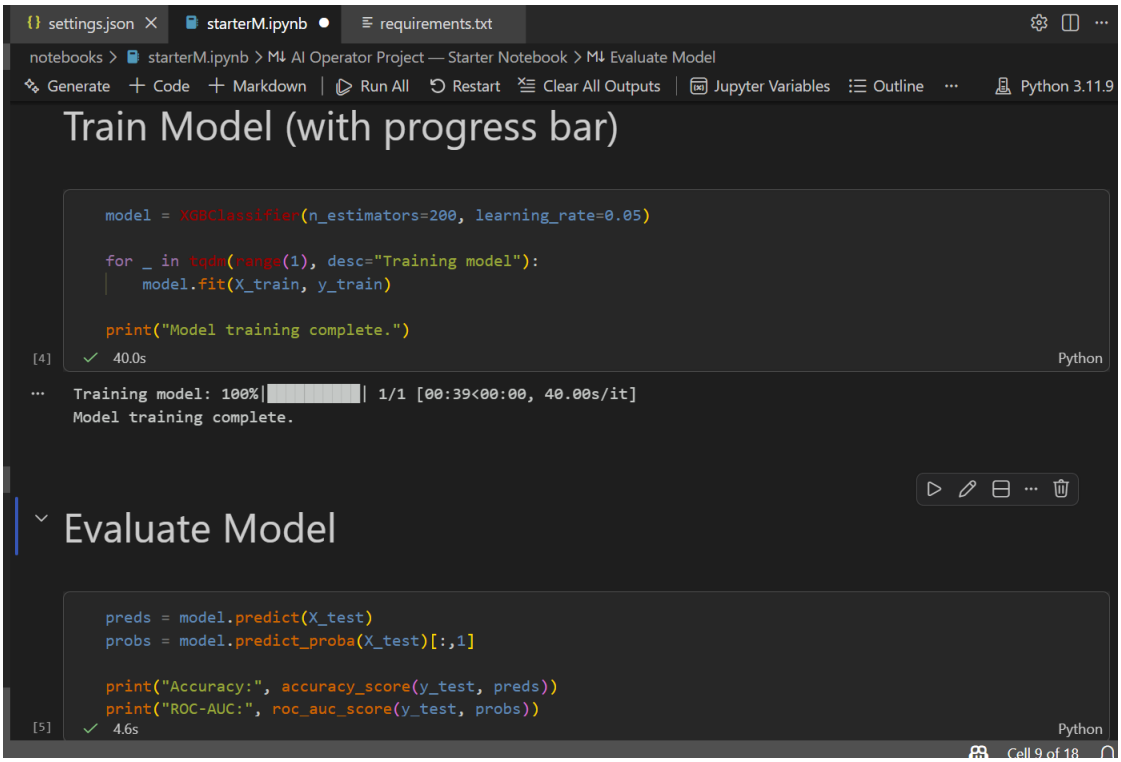

Figure 3: Training model with XGBoost and evaluation with ROC-AUC

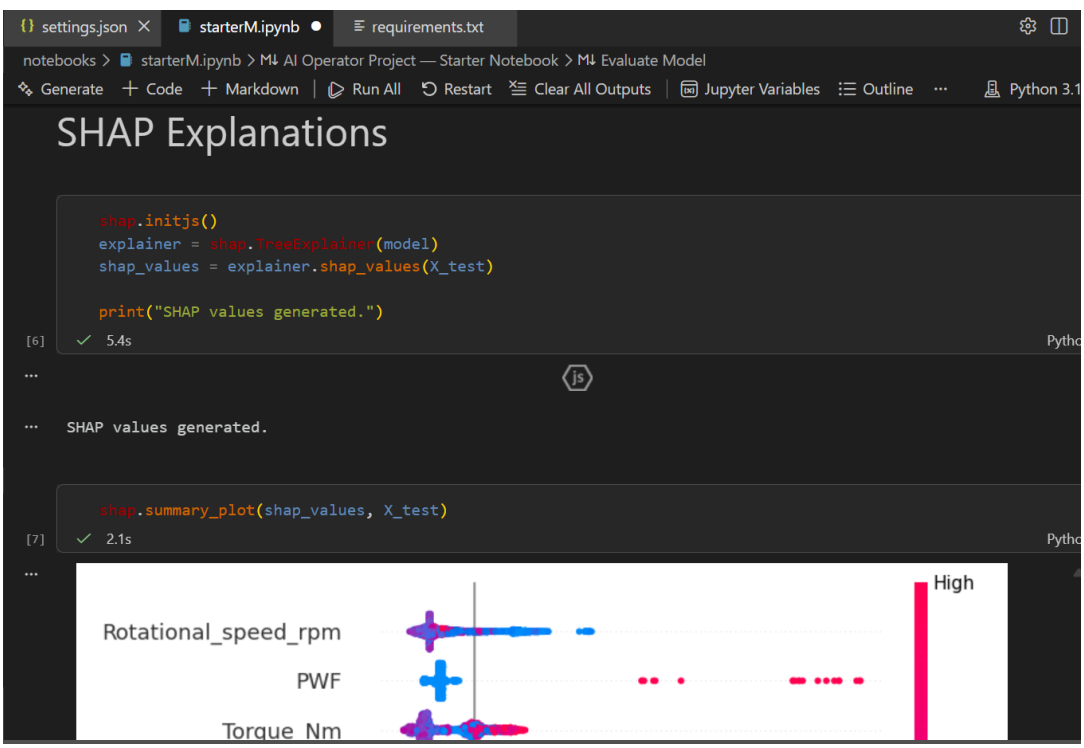

Figure 4: SHAP analysis

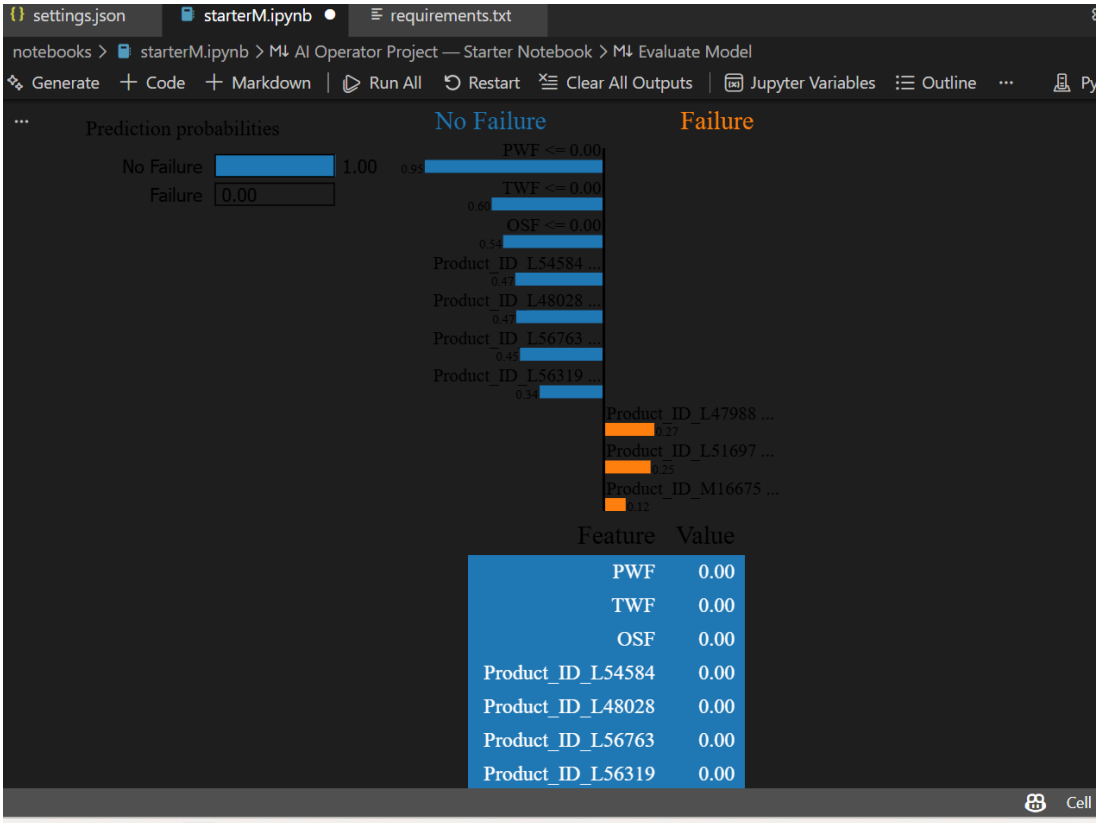

Figure 5: LIME analysis

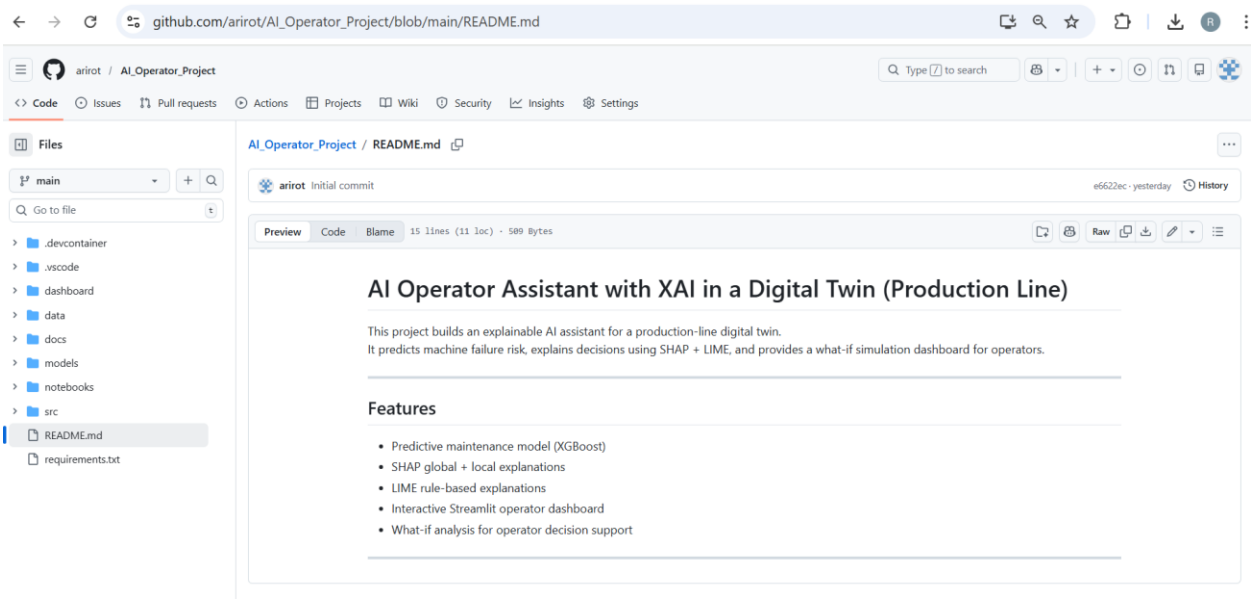

Figure 6: GitHub repository files for Streamlit integration
https://github.com/arirot/AI_Operator_Project/

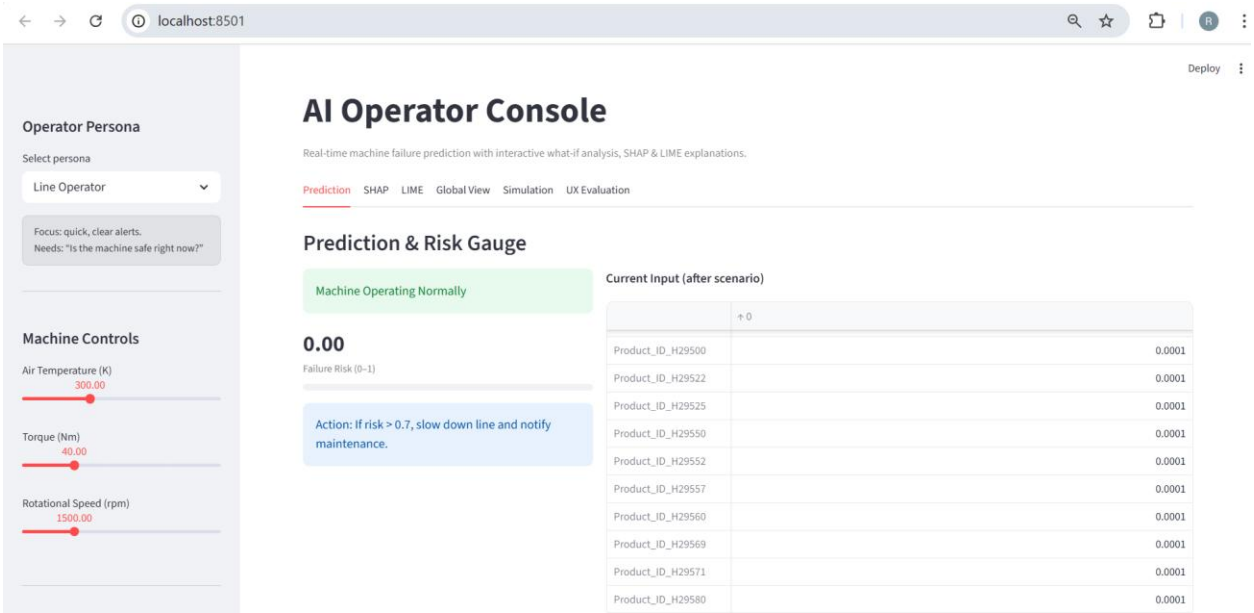

Figure 7: Dashboard hosted locally displaying the prediction and risk gauge.

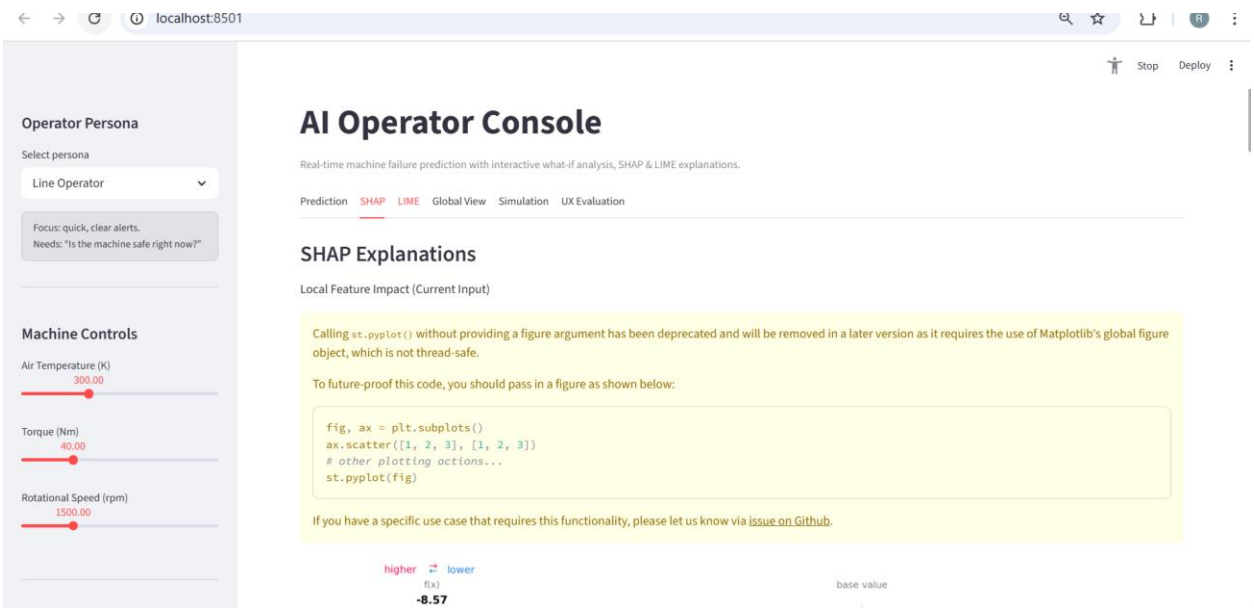

Figure 8: Dashboard hosted locally displaying the SHAP

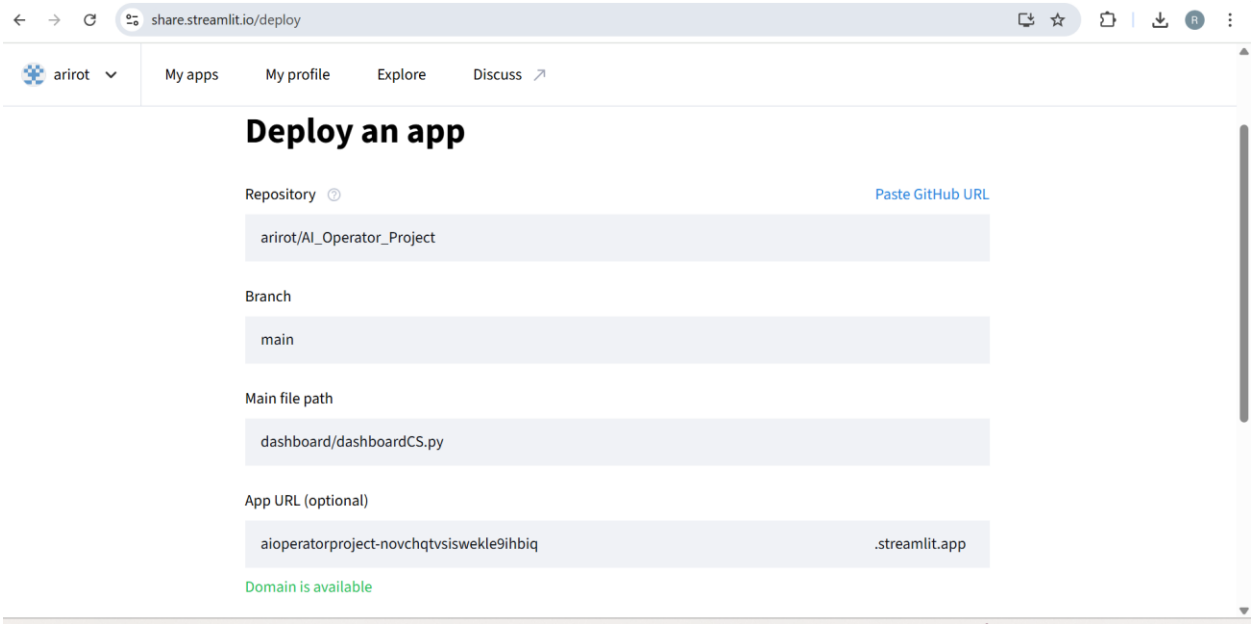

Figure 9: Model deployment on Streamlit Cloud via GitHub

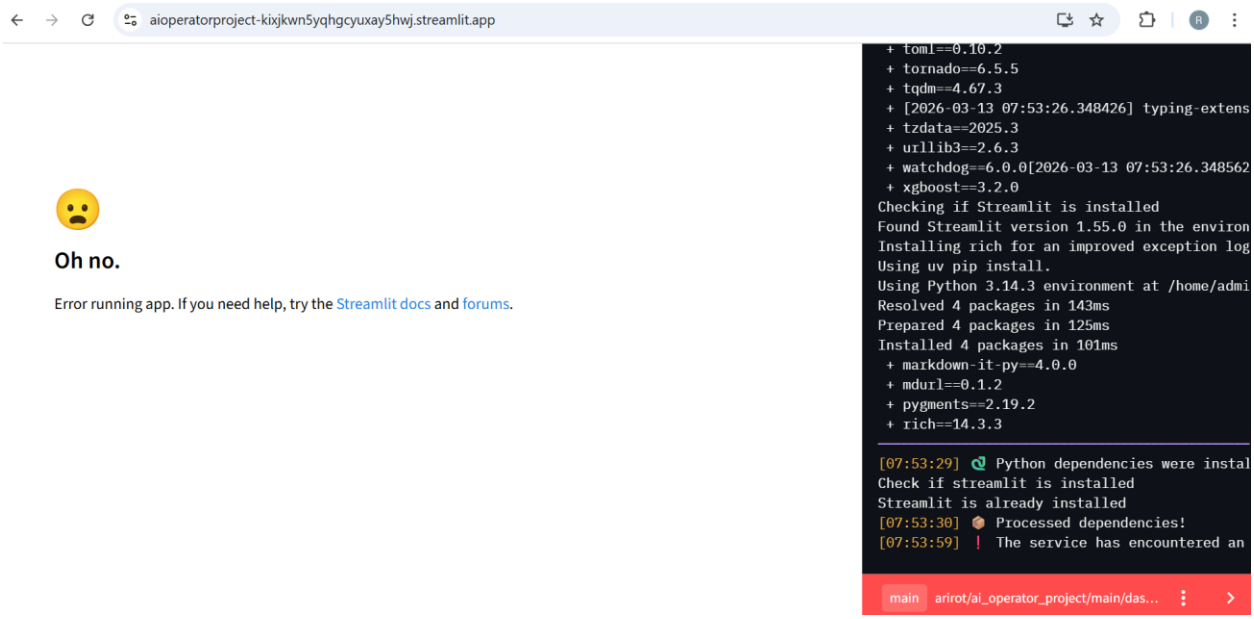

Figure 10: Error encountered while deploying on Streamlit Cloud